



Depth And Alpha

05 Depth And Alpha Artifact

깊이 스탠실 알파 블렌드 문제

게임 프로그래머 | 이창진

| 목차

Chapter 1. Depth-Stencil

Chapter 2. Alpha Blending

Chapter 3. Depth And Alpha Artifact

Chapter 4. Code

Depth-Stencil



정의

픽셀에 대한 깊이 값과 마스크 정보를 저장합니다.

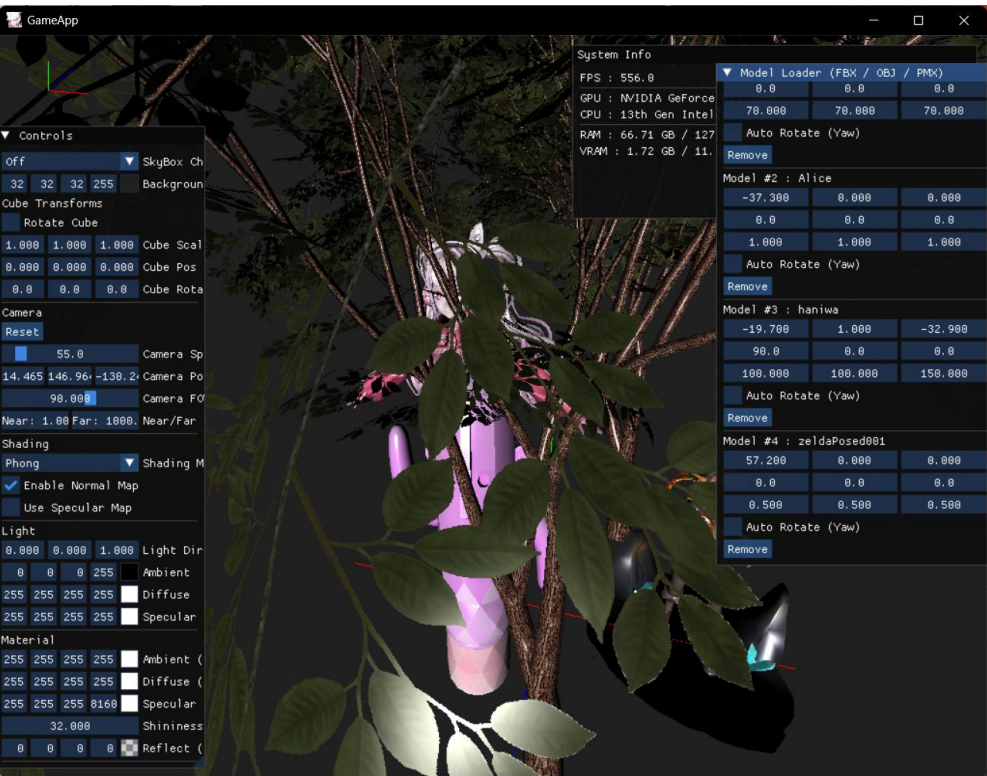
Depth 버퍼는 픽셀의 Z값을 저장하여 앞에 있는 오브젝트가 뒤에 있는 오브젝트를 가리는 기능을 합니다.

픽셀이 렌더링 가능하다고 통과하려면 깊이 테스트에 성공해야 합니다. 이 테스트가 보통 LESS, LEQUAL입니다.

Stencil 버퍼는 특정 픽셀의 처리를 제한하거나 마스크하는 연산에 사용됩니다. 특정 효과, 윤곽선, 영역을 제한하기 등에 사용도 됩니다.

실행순서

템플릿 테스트 -> 깊이 테스트 -> 블렌딩



Alpha Blending



정의

이미지를 합성할 때, 픽셀 색을 혼합해 투명을 표현합니다

알파 테스트인 Clip 조건을 만족하지 못하면 픽셀 자체를
폐기 처리합니다. 이 경우에는 픽셀은 깊이도 색도 기록되지
않습니다

예시

대표적으로 사용되는 곳은 나무의 잎 텍스처 부분, 불의
텍스처 부분 등입니다



Depth And Alpha Artifact

정의

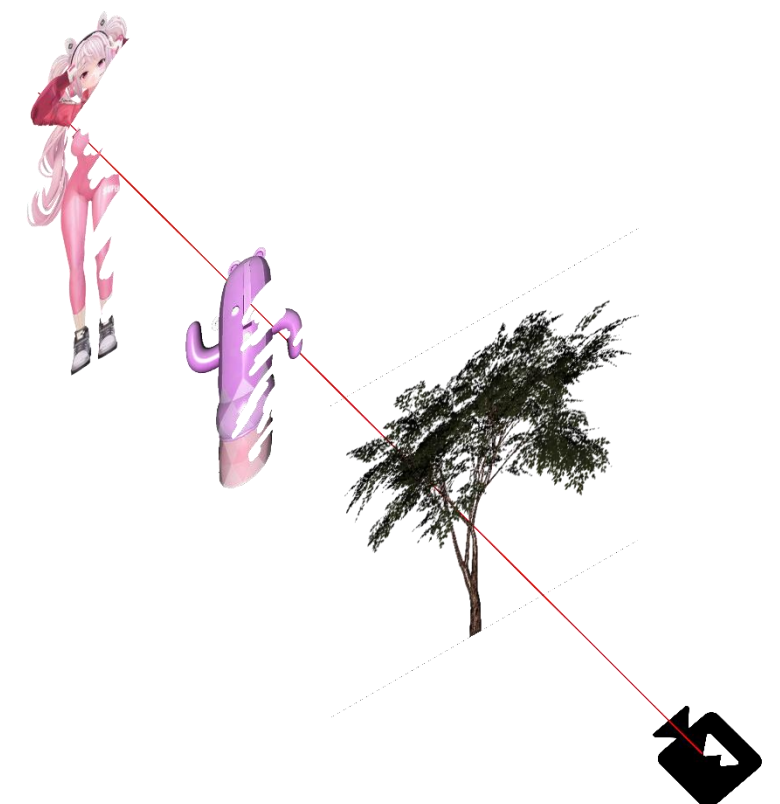
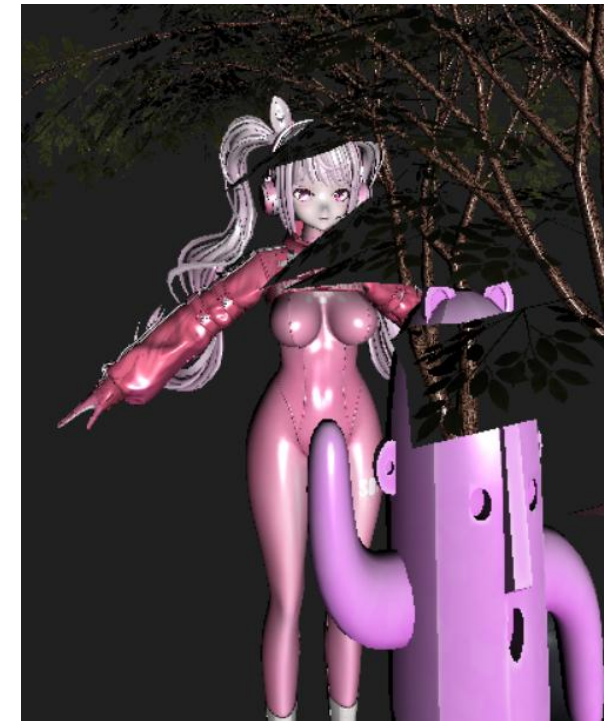
일반적으로 앞 텍스처의 투명 부위는 빛이 통과되어 뒤 오브젝트가 보이고, 불투명한 부분만 가립니다.

그러나 먼저 투명 구간만 골라 깊이를 앞쪽 값(깊이 버퍼에서)으로 채워넣고, 본 패스에서 불투명만 그리면 투명 구간은 이미 깊이가 앞쪽으로 막혀 있으므로, 그 뒤 오브젝트는 깊이 테스트에 실패해 그려지지 않습니다

따라서 그 영역에는 최종적으로 뒤쪽 배경만 남습니다.
깊이/알파 상호작용으로 발생하는 대표적인 문제입니다

실행순서

템플릿 테스트 -> 깊이 테스트 -> 블렌딩





1단계

설명

Tree가 다른 오브젝트보다 먼저 그려지도록 합니다

컬러 쓰기가 OFF된 블렌드 스테이트를 미리 준비해줍니다

상수버퍼의 패딩을 준비합니다. Pixel shader의 분기를 위해서입니다

실행결과

Blend State를 미리 준비해 둔 상태입니다

2단계

설명

투명 프리 패스 (깊이만 기록)

Pixel Shader : $\text{clip}(0.1 - \alpha\text{Tex})$ 로 투명 픽셀만 통과시킵니다

Output Merger : 컬러 쓰기 OFF로 블렌드 (렌더 타겟은 건드리지 않습니다)

실행결과

투명 픽셀 위치의 깊이 버퍼가 가장 가까운 값으로 채워집니다

3단계

설명

PixelShader : $\text{clip}(\alpha\text{Tex} - 0.1)$ 로 불투명 픽셀만 통과시킵니다

Output Merger : 보통의 알파 블렌딩/ 깊이 테스트를 그대로 진행합니다

실행결과

투명 구간은 색을 그리지 않지만, 이미 2단계에서 깊이가 막혀 뒤 오브젝트는 깊이 테스트에 실패합니다. 즉 배경만 보입니다

Code – 1단계



1단계

설명

Tree가 다른 오브젝트보다 먼저 그려지도록 합니다

컬러 쓰기가 OFF된 블렌드 스테이트를 미리 준비해줍니다

상수버퍼의 패딩을 준비합니다. Pixel shader의 분기를 위해서입니다

실행결과

Blend State를 미리 준비해 둔 상태입니다

cpp

```
// 알파 블렌딩 상태 생성 SrcAlpha/InvSrcAlpha
D3D11_BLEND_DESC bd{};
bd.AlphaToCoverageEnable = FALSE;
bd.IndependentBlendEnable = FALSE;
D3D11_RENDER_TARGET_BLEND_DESC& rt = bd.RenderTarget[0];
rt.BlendEnable = TRUE;
rt.SrcBlend = D3D11_BLEND_SRC_ALPHA;
rt.DestBlend = D3D11_BLEND_INV_SRC_ALPHA;
rt.BlendOp = D3D11_BLEND_OP_ADD;
rt.SrcBlendAlpha = D3D11_BLEND_ONE;
rt.DestBlendAlpha = D3D11_BLEND_INV_SRC_ALPHA;
rt.BlendOpAlpha = D3D11_BLEND_OP_ADD;
rt.RenderTargetWriteMask = D3D11_COLOR_WRITE_ENABLE_ALL;
HR_T(m_ ->m_pDevice->CreateBlendState(&bd, &m_ ->m_pAlphaBlendState));

// 컬러 마스크 OFF 깊이 프리패스용
D3D11_BLEND_DESC bdMask{};
bdMask.AlphaToCoverageEnable = FALSE;
bdMask.IndependentBlendEnable = FALSE;
D3D11_RENDER_TARGET_BLEND_DESC& rtM = bdMask.RenderTarget[0];
rtM.BlendEnable = FALSE;
rtM.RenderTargetWriteMask = 0; // RGBA 모두 비활성
HR_T(m_ ->m_pDevice->CreateBlendState(&bdMask, &m_ ->m_pColorMaskNone));
```

Pixel Shader

```
cbuffer ConstantBuffer : register b0
{
    matrix g_World // 매트릭스는 float4x4로
    대체 될 수 있습니다. 행이 없으면 매트릭스는 기본적으로 열
    매트릭스로 기본값을 얻습니다.
    matrix g_View // 행 매트릭스를 대표하기
    전에 행을 추가 할 수 있습니다.
    matrix g_Proj // 이 튜토리얼은 향후 기본
    열 마스터 매트릭스를 사용하지만 매트릭스는 C ++ 코드
    측면에서 미리 변환해야 합니다.
    matrix g_WorldInvTranspose // 월드 행렬의
    역전치 행렬입니다. 조명 계산에 필요합니다. 이번
    프로젝트에 추가되었습니다.

    Material g_Material // 머티리얼 구조체
    DirectionalLight g_DirLight
    float3 g_EyePosW
    float g_Pad
    int g_ShadingMode //
    0:Phong,1:Blinn,2:Lambert,3:Unlit,4:TextureOnly
    float3 g_Pad2
    int g_EnableNormalMap // 0: off, 1: on
    float3 g_Pad3
    int g_UseSpecularMap // 0: off, 1: on
    float3 g_Pad4
}
```

Code – 2단계



2단계

설명

투명 프리 패스 (깊이만 기록)

Pixel Shader : clip(0.1 - alphaTex) 로
투명 픽셀만 통과시킵니다

Output Merger : 컬러 쓰기 OFF로 블렌드
(렌더 타겟은 건드리지 않습니다)

실행결과

투명 픽셀 위치의 깊이 버퍼가 가장 가까운
값으로 채워집니다

cpp

```
// 1. 깊이 프리패스: 투명 영역만 Z 채움 (g_Pad==9)
ConstantBuffer pre = cb; pre.pad = 9.0f;
D3D11_MAPPED_SUBRESOURCE mappedPre;
HR_T(m_->m_pDeviceContext->Map(m_->m_pConstantBuffer, 0,
D3D11_MAP_WRITE_DISCARD, 0, &mappedPre));
memcpy_s(mappedPre.pData, sizeof(ConstantBuffer), &pre,
sizeof(ConstantBuffer));
m_->m_pDeviceContext->Unmap(m_->m_pConstantBuffer, 0);
m_->m_pDeviceContext->VSSetConstantBuffers(0, 1, &m_-
>m_pConstantBuffer);
m_->m_pDeviceContext->PSSetConstantBuffers(0, 1, &m_-
>m_pConstantBuffer);

// 2. 컬러 쓰기 OFF (깊이만 기록)
m_->m_pDeviceContext->OMSetBlendState(m_->m_pColorMaskNone,
nullptr, 0xFFFFFFFF);
m_->m_pDeviceContext->DrawIndexed(sub.count, sub.start, 0);

m_->m_pDeviceContext->OMSetBlendState(nullptr, nullptr, 0xFFFFFFFF);
```

Pixel Shader

```
// 알파 컷아웃 조명 모드에서만 적용
float4 textureColor = g_DiffuseMap.Sample(g_Sam,
pIn.tex);
float alphaTex = textureColor.a *
g_Material.diffuse.a;
// 기본: 알파 컷아웃 적용
// g_Pad==8 : 투명 픽셀만 통과. 깊이만 기록, 컬러는
외부에서 마스
if (abs(g_Pad - 8.0f) < 1e-3)
{
    // no clip
}
else if (abs(g_Pad - 9.0f) < 1e-3)
{
    // 투명 영역만 남기기: alpha < 0.1만 통과
    clip(0.1f - alphaTex);
}
else
{
    clip(alphaTex - 0.1f);
}
```


Code – 3단계



3단계

설명

PixelShader : clip(alphaTex - 0.1)로 불투명 픽셀만 통과시킵니다

Output Merger : 보통의 알파 블렌딩/깊이 테스트를 그대로 진행합니다

실행결과

투명 구간은 색을 그리지 않지만, 이미 2단계에서 깊이가 막혀 뒤 오브젝트는 깊이 테스트에 실패합니다. 즉 배경만 보입니다

cpp

```
// 3. 본 패스: clip 활성화로 실제 컬러 출력 투명은 버려짐. 뒤 오브젝트는 이미 깊이로 가려짐. 배경만 보임
HR_T(m_->m_pDeviceContext->Map(m_->m_pConstantBuffer, 0, D3D11_MAP_WRITE_DISCARD, 0, &mappedPre));
memcpy_s(mappedPre.pData, sizeof(ConstantBuffer), &cb, sizeof(ConstantBuffer));
m_->m_pDeviceContext->Unmap(m_->m_pConstantBuffer, 0);
m_->m_pDeviceContext->VSSetConstantBuffers(0, 1, &m_->m_pConstantBuffer);
m_->m_pDeviceContext->PSSetConstantBuffers(0, 1, &m_->m_pConstantBuffer);
```

Pixel Shader

```
// 알파 컷아웃 조명 모드에서만 적용
float4 textureColor = g_DiffuseMap.Sample(g_Sam, pIn.tex);
float alphaTex = textureColor.a * g_Material.diffuse.a;
// 기본: 알파 컷아웃 적용
// g_Pad==8: 투명 픽셀만 통과. 깊이만 기록, 컬러는 외부에서 마스크
if (abs(g_Pad - 8.0f) < 1e-3)
{
    // no clip
}
else if (abs(g_Pad - 9.0f) < 1e-3)
{
    // 투명 영역만 남기기: alpha < 0.1만 통과
    clip(0.1f - alphaTex);
}
else
{
    clip(alphaTex - 0.1f);
}
```

감사합니다.